

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет Прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
«РАБОТА С БД В СУБД MONGODB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся	Матюков Дмитрий Владимирович
Группа	K3240
Факультет	Прикладной информатики
Образовательная программа	Мобильные и сетевые технологии
Направление подготовки	09.03.03 Прикладная информатика
Преподаватели:	Горова Марина Михайловна Белов Александр Олегович

Санкт-Петербург
2026

Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание

1. Установить СУБД MongoDB Community Server.
2. Создать базу данных с использованием консоли.
3. Создать коллекции по заданию
4. Выполнить CRUD-операции, задания по вставке данных и выборке
5. Выполнить задание по использованию курсора, агрегированию запросов, изменению данных
6. Выполнить задания по работе с индексами

Выполнение задания

Сначала нужно запустить сервер mongod.exe

```
D:\Program Files\MongoDB\Server\8.3\bin>mongod.exe --dbpath D:\mongodb_data
{"t":{"$date":"2026-06-08T13:38:50.787+03:00"},"s":"I", "c":"-", "id":8991200, "ctx":
:"thread1", "msg":"Shuffling initializers", "attr":{"seed":715532708}}
{"t":{"$date":"2026-06-08T13:38:50.840+03:00"},"s":"I", "c":"CONTROL", "id":97374, "ctx":
:"thread1", "msg":"Automatically disabling TLS 1.0 and TLS 1.1, to force-enable TLS 1.1 speci
fy --sslDisabledProtocols 'TLS1_0'; to force-enable TLS 1.0 specify --sslDisabledProtocols '
none'"}
none'"}

```

Затем во втором окне Командной строки я запустил клиент mongosh.exe

```
D:\Program Files\MongoDB\Server\8.3\bin>mongosh.exe
Current Mongosh Log ID: 6a269952b208d215c3abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.8.3
MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017

D:\Program Files\MongoDB\Server\8.3\bin>mongosh.exe
Current Mongosh Log ID: 6a269c13a039c06b21abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.8.3
Using MongoDB:      8.3.2
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-06-08T13:38:51.586+03:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
2026-06-08T13:38:51.587+03:00: This server is bound to localhost. Remote systems will be unable to connect to this se
rver. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --
bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable
this warning
-----
test> |
```

2 CRUD-ОПЕРАЦИИ В СУБД MONGODB. ВСТАВКА ДАННЫХ. ВЫБОРКА ДАННЫХ

2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1:

1) Создайте базу данных *learn*.

2) Заполните коллекцию единорогов *unicorns*:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600,
gender: 'm', vampires: 63});

db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});

db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182});

db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender:
'm', vampires: 99});

db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'],
weight: 550, gender: 'f', vampires: 80});

db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});

db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});

db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
gender: 'm', vampires: 2});

db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601,
gender: 'f', vampires: 33});

db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650,
gender: 'm', vampires: 54});

db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,
gender: 'f'});
```

```
test> use learn
switched to db learn
learn> show dbs
admin      40.00 KiB
config     72.00 KiB
local      64.00 KiB
myNewDB    40.00 KiB
```

```
| db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 42
1, gender: 'm', vampires: 2});
| db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight:
601, gender: 'f', vampires: 33});
| db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight:
650, gender: 'm', vampires: 54});
| db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540
, gender: 'f'});
|
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insert
Many, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a269de2a039c06b21abc11e') }
}
learn> |
```

Предупреждение *DeprecationWarning* говорит о том, что метод *insert()* считается устаревшим, но в текущей версии он всё ещё работает

```
test> use learn
switched to db learn
learn> show dbs
admin      40.00 KiB
config     96.00 KiB
learn      40.00 KiB
local      64.00 KiB
myNewDB    40.00 KiB
learn> |
```

После вставки данных появилась база данных

3) Используя второй способ, вставьте в коллекцию единорогов документ:

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm',
vampires: 165}
```

```
learn> var dunx = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,
gender: 'm', vampires: 165};
| db.unicorns.insert(dunx);
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insert
Many, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a269fb463c46709fbabc114') }
}
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a269de2a039c06b21abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
```

4) Проверьте содержимое коллекции с помощью метода *find*.

Вывелось 12 документов, в том числе тот, который вставил вторым способом

```

{
  _id: ObjectId('6a269de2a039c06b21abc11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a269fb463c46709fbabc114'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
learn> |

```

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

Практическое задание 2.2.1:

1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

Список самцов/самок:

```

learn> db.unicorns.find({ gender: 'm' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc114'),
    name: 'Rorny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc116'),
    name: 'Unicrow',
    loves: [ 'energyon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc117'),
    name: 'Noooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11a'),
    name: 'Denny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a269fb463c46709fbabc114'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> |

```

```

learn> db.unicorns.find({ gender: 'f' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |

```

Вывести список самок (gender: 'f') с ограничением первых трёх особей и сортировкой по имени:

```
learn> db.unicorns.find({ gender: 'f' }).sort({ name: 1 }).limit(3)
[
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

Найти всех самок, которые любят carrot (поле loves должно содержать строку "carrot"):

```
learn> db.unicorns.find({ gender: 'f', loves: 'carrot' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Ограничьте этот список первой особью с помощью функций `findOne` и `limit`:

```
learn> db.unicorns.findOne({ gender: 'f', loves: 'carrot' })
{
  _id: ObjectId('6a269de2a039c06b21abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({ gender: 'f', loves: 'carrot' }).limit(1)
[
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({ gender: 'm' }, { loves: 0, _id: 0 })
[
  { name: 'Horny', weight: 600, gender: 'm', vampires: 63 },
  { name: 'Unicrom', weight: 984, gender: 'm', vampires: 182 },
  { name: 'Roooooodles', weight: 575, gender: 'm', vampires: 99 },
  { name: 'Kenny', weight: 690, gender: 'm', vampires: 39 },
  { name: 'Raleigh', weight: 421, gender: 'm', vampires: 2 },
  { name: 'Pilot', weight: 650, gender: 'm', vampires: 54 },
  { name: 'Dunx', weight: 704, gender: 'm', vampires: 165 }
]
learn> |
```

Если нужно отсортировать ограниченную коллекцию, то можно воспользоваться параметром `$natural`. Этот параметр позволяет задать сортировку: документы передаются в том порядке, в каком они были добавлены в коллекцию, либо в обратном порядке.

Например, отобразить последние пять документов:

```
> db.users.find().sort({ $natural: -1 }).limit(5)
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find({ gender: 'm' }, { loves: 0, _id: 0 }).sort({ $natural: -1 })
[
  { name: 'Dunx', weight: 704, gender: 'm', vampires: 165 },
  { name: 'Pilot', weight: 650, gender: 'm', vampires: 54 },
  { name: 'Raleigh', weight: 421, gender: 'm', vampires: 2 },
  { name: 'Kenny', weight: 690, gender: 'm', vampires: 39 },
  { name: 'Roooooodles', weight: 575, gender: 'm', vampires: 99 },
  { name: 'Unicrom', weight: 984, gender: 'm', vampires: 182 },
  { name: 'Horny', weight: 600, gender: 'm', vampires: 63 }
]
learn> |
```

Для работы с массивами используется оператор `$slice`. Он является в некотором роде комбинацией функций `limit` и `skip`.

Практическое задание 2.2.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, { loves: { $slice: 1 }, _id: 0 })
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: [ 'apple' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Kenny',
    loves: [ 'grape' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Raleigh',
    loves: [ 'apple' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Leia',
    loves: [ 'apple' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Pilot',
    loves: [ 'apple' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    name: 'Nimue', loves: [ 'grape' ], weight: 540, gender: 'f' },
  {
    name: 'Dunx',
    loves: [ 'grape' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> |
```

В качестве селектора могут выступать не только строки, но и регулярные выражения. Например, найти все документы, в которых значение ключа name начинается с буквы T:

```
> db.users.find({name:/T\w+/i})
```


2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 700 } }, { _id: 0 })
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({ gender: 'm', weight: { $gte: 500 }, loves: { $all: [ 'grape', 'lemon' ] } }, { _id: 0 })
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({ vampires: { $exists: false } })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({ gender: 'm' }, { name: 1, loves: { $slice: 1 }, _id: 0 }).sort({ name: 1 })
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB. ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

- *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a26adfc63c46709fbabc115'),
    '1': ObjectId('6a26adfc63c46709fbabc116'),
    '2': ObjectId('6a26adfc63c46709fbabc117')
  }
}
learn> |
```

- Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({ "mayor.party": "I" }, { name: 1, mayor: 1, _id: 0 })
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> |
```

- Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({ "mayor.party": { $exists: false } }, { name: 1, mayor:
1, _id: 0 })
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

Практическое задание 3.1.2:

- 1) Сформировать функцию для вывода списка самцов единорогов.

```
learn> function listMales() {
|   return db.unicorns.find({ gender: "m" })
| }
[Function: listMales]
learn> |
```

- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
- 3) Вывести результат, используя `forEach`.

```
learn> var maleCursor = db.unicorns.find({ gender: "m" }).sort({ name: 1 }).limit(2)

learn> maleCursor.forEach(function(doc) {
|   print(doc.name)
| })
Dunx
Horny

learn> |
```

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.count({ gender: "f", weight: { $gte: 500, $lte: 600 } })
DeprecationWarning: Collection.count() is deprecated. Use countDocuments or estimatedDocumentCount.
2
learn> db.unicorns.countDocuments({ gender: "f", weight: { $gte: 500, $lte: 600 } })
2
learn> |
```

Предупреждение `DeprecationWarning` говорит о том, что метод `count()` считается устаревшим, но в текущей версии он всё ещё работает

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> |
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([ { $group: { _id: "$gender", count: { $sum: 1 } } } ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> |
```

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1:

- *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],  
weight: 340, gender: 'm'})
```

```
learn> db.unicorns.save({ name: 'Barney', loves: ['grape'], weight: 340, gender: 'm' })  
TypeError: db.unicorns.save is not a function  
learn> |
```

Ошибка возникает из-за того, что в современной версии mongosh метод save() больше не поддерживается для коллекций. Вместо него я использовал insertOne() .

- *Проверить содержимое коллекции unicorns .*

```
learn> db.unicorns.insertOne({ name: 'Barney', loves: ['grape'], weight: 340, gender: 'm' })  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a26b43c63c46709fbabc118')  
}  
learn> db.unicorns.find({ name: 'Barney' })  
[  
  {  
    _id: ObjectId('6a26b43c63c46709fbabc118'),  
    name: 'Barney',  
    loves: [ 'grape' ],  
    weight: 340,  
    gender: 'm'  
  }  
]  
learn> |
```

Практическое задание 3.3.2:

- *Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.*
- *Проверить содержимое коллекции unicorns .*

```
learn> db.unicorns.updateOne({ name: 'Ayna' }, { $set: { weight: 800, vampires: 51 } })  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
learn> db.unicorns.find({ name: 'Ayna' })  
[  
  {  
    _id: ObjectId('6a269de2a039c06b21abc119'),  
    name: 'Ayna',  
    loves: [ 'strawberry', 'lemon' ],  
    weight: 800,  
    gender: 'f',  
    vampires: 51  
  }  
]  
learn> |
```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne({ name: 'Raleigh' }, { $push: { loves: 'redbull' } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: 'Raleigh' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> |
```

Практическое задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateMany({ gender: 'm' }, { $inc: { vampires: 5 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
```

```
learn> db.unicorns.find({ gender: 'm' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 7
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  },
  {
    _id: ObjectId('6a269fb463c46709fbabc114'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 170
  },
  {
    _id: ObjectId('6a26b43c63c46709fbabc118'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm',
    vampires: 5
  }
]
learn> |
```

Действительно, у каждого самца стало на 5 больше вампиров

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции `towns`.

```

learn> db.towns.updateOne({ name: 'Portland' }, { $unset: { party: "" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.towns.find({ name: 'Portland' })
[
  {
    _id: ObjectId('6a26adfc63c46709fbabc117'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.updateOne({ name: 'Portland' }, { $unset: { "mayor.party": "" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({ name: 'Portland' })
[
  {
    _id: ObjectId('6a26adfc63c46709fbabc117'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |

```

Команда сначала не удалила поле `party`, потому что я указал `{ $unset: { party: "" } }`, а поле `party` находится внутри вложенного документа `mayor`, а не на корневом уровне.

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
2. Проверить содержимое коллекции *unicorns*.

```

learn> db.unicorns.updateOne({ name: 'Pilot' }, { $push: { loves: 'chocolate' } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: 'Pilot' })
[
  {
    _id: ObjectId('6a269de2a039c06b21abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> |

```


Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateOne({ name: 'Aurora' }, { $addToSet: { loves: { $each: ['sugar', 'lemon'] } } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: 'Aurora'})
[
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |
```

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Практическое задание 3.4.1:

- Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
popujatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: ["phil the groundhog"],
mayor: {
  name: "Jim Wehrle"
}}
```

```
{name: "New York",
popujatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}
```

```
{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

- Удалите документы с беспартийными мэрами.

- Проверьте содержание коллекции.
- Очистите коллекцию.
- Просмотрите список доступных коллекций.

```
learn> db.towns.deleteMany({ "mayor.party": { $exists: false } })
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a26bcb863c46709fbabc11a'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a26bcb863c46709fbabc11b'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> db.towns.find()

learn> |
```

Теперь коллекция *towns* пустая

4 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

4.1 ССЫЛКИ В БД

Практическое задание 4.1.1:

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.habitats.insertMany([
  { _id: "forest", name: "Enchanted Forest", description: "Dense magical forest with glowing mushrooms" },
  { _id: "meadow", name: "Sunlit Meadow", description: "Open field with colorful flowers and butterflies" },
  { _id: "mountains", name: "Crystal Mountains", description: "High peaks with sparkling crystals" }
]);
learn> db.habitats.insertMany([
  { _id: "forest", name: "Enchanted Forest", description: "Dense magical forest with glowing mushrooms" },
  { _id: "meadow", name: "Sunlit Meadow", description: "Open field with colorful flowers and butterflies" },
  { _id: "mountains", name: "Crystal Mountains", description: "High peaks with sparkling crystals" }
]);
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'meadow', '2': 'mountains' }
}
```

2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```
learn> db.unicorns.updateOne({ name: "Horny" }, { $set: { habitat: { $ref: "habitats", $id: "forest" } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({ name: "Aurora" }, { $set: { habitat: { $ref: "habitats", $id: "meadow" } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({ name: "Unicrom" }, { $set: { habitat: { $ref: "habitats", $id: "mountains" } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> |
```

3) Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find({ name: { $in: ["Horny", "Aurora", "Unicrom"] } }, { name: 1, habitat: 1 });
[
  {
    _id: ObjectId('6a269de2a039c06b21abc114'),
    name: 'Horny',
    habitat: DBRef('habitats', 'forest')
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc115'),
    name: 'Aurora',
    habitat: DBRef('habitats', 'meadow')
  },
  {
    _id: ObjectId('6a269de2a039c06b21abc116'),
    name: 'Unicrom',
    habitat: DBRef('habitats', 'mountains')
  }
]
learn> |
```

4) Содержание коллекции единорогов unicorns:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
```

```
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```

db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight:
690, gender: 'm', vampires: 39});

db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight:
421, gender: 'm', vampires: 2});

db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'],
weight: 601, gender: 'f', vampires: 33});

db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'],
weight: 650, gender: 'm', vampires: 54});

db.unicorns.insert ({name: 'Nimue', loves: ['grape', 'carrot'], weight:
540, gender: 'f'});

db.unicorns.insert {name: 'Dunx', loves: ['grape', 'watermelon'],
weight: 704, gender: 'm', vampires: 165}

```

4.2 НАСТРОЙКА ИНДЕКСОВ

Когда коллекции содержат миллионы документов, а нужно сделать выборку по определенному полю, то поиск нужных данных может занять некоторое время, которое может оказаться критичным для задачи. В этом случае могут помочь индексы.

Индексы позволяют упорядочить данные по определенному полю, что впоследствии ускорит поиск. Например, если в приложении или задаче, как правило, выполняется поиск по полю `name`, то можно индексировать коллекцию по этому полю:

```
> db.users.ensureIndex({"name": 1})
```

Значение 1 сообщает, что индекс следует выполнить в порядке возрастания.

Примечание. Содержание коллекции users:

```

> db.users.insert({"name": "Tom", "age": 28, languages: ["english",
"spanish"]})

> db.users.insert({"name": "Bill", "age": 32, languages: ["english",
"french"]})

> db.users.insert({"name": "Tom", "age": 32, languages: ["english",
"german"]})

```

Таким образом с помощью метода `ensureIndex` устанавливается индекс по полю `name`. MongoDB позволяет установить до 64 индексов на одну коллекцию.

Создать индекс можно также с помощью метода `createIndex()`.

Если нужно просто определить индекс для коллекции, например, `db.users.ensureIndex({"name": 1})`, то можно добавлять в коллекцию документы с одинаковым значением ключа `name`. Однако, если потребуется, чтобы в коллекцию можно было добавлять документ с одним и тем же значением ключа только один раз, нужно установить флаг `unique`:

```
> db.users.ensureIndex({"name": 1}, {"unique": true})
```

Теперь, если попытаться добавить в коллекцию два документа с одним и тем же значением `name`, то получится ошибка.

Практическое задание 4.2.1:

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.
2. Содержание коллекции единорогов `unicorns`:

```
db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47),  
loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0),  
loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22,  
10), loves: ['energon', 'redbull'], weight: 984, gender: 'm',  
vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979, 7, 18,  
18, 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1),  
loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f',  
vampires:80});
```

```
db.unicorns.insert({name:'Ayna', dob: new Date(1998, 2, 7, 8, 30),  
loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires:  
40});
```

```
db.unicorns.insert({name:'Kenny', dob: new Date(1997, 6, 1, 10, 42),  
loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57),  
loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53),  
loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires:  
33});
```

```
db.unicorns.insert({name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3),  
loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires:  
54});
```

```
db.unicorns.insert ({name: 'Nimue', dob: new Date(1999, 11, 20, 16,  
15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert {name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18),  
loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires:  
165
```

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })  
name_1  
learn> |
```

Вывод `name_1` – это название созданного индекса

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

Практическое задание 4.3.1:

1) Получите информацию о всех индексах коллекции `unicorns`.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> |
```

2) Удалите все индексы, кроме индекса для идентификатора.

3) Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

4.4 ПЛАН ЗАПРОСА

С помощью метода `explain()` можно получить информацию о выполнении запроса. Метод `explain()` возвращает JSON-структуру с планом выполнения запроса.

Для детализации плана запроса нужно указать параметр `"executionStats"` для метода `explain()`:

```
db.users.explain("executionStats").find({})
```

Практическое задание 4.4.1:

1) Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
learn> for (var i = 0; i < 100000; i++) {db.numbers.insertOne({ value: i });
}
{
  acknowledged: true,
  insertedId: ObjectId('6a27fd3dc019d654fdb054f3')
}
```

2) Выберите последних четыре документа.

```
learn> db.numbers.find().sort({ value: -1 }).limit(4);
[
  { _id: ObjectId('6a27fd3dc019d654fdb054f3'), value: 99999 },
  { _id: ObjectId('6a27f8dcc019d654fdaece53'), value: 99999 },
  { _id: ObjectId('6a27f82ac019d654fdad47b3'), value: 99999 },
  { _id: ObjectId('6a27f8dcc019d654fdaece52'), value: 99998 }
]
```

- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

```
learn> db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4);
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShaneHash: 'AE5753F2'
```

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 587,
    totalKeysExamined: 0,
    totalDocsExamined: 300000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
```

- 4) Создайте индекс для ключа `value`.

```
learn> db.numbers.createIndex({ value: 1 })
value_1
learn> |
```

- 5) Получите информацию о всех индексах коллекции `numbers`.

```
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> |
```

- 6) Выполните запрос 2.

```
learn> db.numbers.find().sort({ value: -1 }).limit(4);
[
  { _id: ObjectId('6a27fd3dc019d654fdb054f3'), value: 99999 },
  { _id: ObjectId('6a27f8dcc019d654fdaece53'), value: 99999 },
  { _id: ObjectId('6a27f82ac019d654fdad47b3'), value: 99999 },
  { _id: ObjectId('6a27fd3dc019d654fdb054f2'), value: 99998 }
]
learn> |
```

- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

```
learn> db.numbers.explain("executionStats").find().sort({ value: -1 }).limit
(4);
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {}
  }
}
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 1,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5
  }
}
```

- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Запрос с использованием индекса значительно эффективнее, так как время выполнения сокращается в десятки или сотни раз (587 мс -> 1 мс). Индексы критически важны для производительности на больших объёмах данных.

Вывод

Овладел практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.